project, but it’s not ideal to have the test data feature enabled by default. What if you forget to overwrite it in production? A better option is configuring it specifically for the local development environment when running the bootRun task. You can achieve that by adding the following code to the build.gradle file.

Listing 4.7 Defining the active profiles for the development environment

```
bootRun {
  systemProperty 'spring.profiles.active', 'testdata'
}
```

Let’s verify that it works. Build and run the application (`./gradlew bootRun`). You’ll see a message in the application logs listing all the active profiles (in this case, it’s just `testdata`, but there can be more), as shown in figure 4.5.

```
...Starting CatalogServiceApplication using Java 17
...The following 1 profile is active: "testdata"
...Tomcat initialized with port(s): 9001 (http)
...Starting service [Tomcat]
...Starting Servlet engine: [Apache Tomcat/9.0.65]
...Initializing Spring embedded WebApplicationContext
...Root WebApplicationContext: initialization completed in 276 ms
...Tomcat started on port(s): 9001 (http) with context path ''
...Started CatalogServiceApplication in 0.529 seconds (JVM running for 0.735)
```

Figure 4.5 The logs for Catalog Service when the “testdata” profile is active